

Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 18

Recap and outline

- Move from global to local perspective
- Feature attribution
- Feature attribution (local) vs feature selection (global)
- Examples

Setup

Data: (p features and n samples)

$$(\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n$$

Model/function:

$$\mathbb{E}[Y|X_1, X_2, \dots, X_p] = \sum_{j=1}^p \beta_j X_j$$

Prediction algorithm/estimator:

$$\widehat{f}(\cdot) = \mathcal{A}((\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n) =$$

- For example, $\mathcal{A}(\cdot)$ may be linear regression, decision tree/random forest, neural network or others
- For a *local* or *specific* instance/example with features $\tilde{X} = (\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$, prediction is $\widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$
- **Question:** How much does each feature contribute to local prediction $\widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$?

Feature attribution

- How do we assign an attribution ϕ_j for each feature $\tilde{X}_j$ that captures importance for the *individual prediction*?
- Key property (completeness/efficiency):

$$\sum_{j=1}^p \phi_j = \widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$$

- Prediction can be decomposed into sum of p feature attributions
- Other properties often introduced

Feature attribution for linear regression

- For individual with features $\tilde{X} = (\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$
- Overall prediction is $\widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p) = \sum_{j=1}^p \widehat{\beta}_j \tilde{X}_j$
- How do we assign an attribution ϕ_j for each feature $\tilde{X}_j$ that captures importance for the *individual prediction*?
- **Feature attribution:** $\phi_j = \widehat{\beta}_j \tilde{X}_j$
- Key property (completeness/efficiency):

$$\sum_{j=1}^p \phi_j = \widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$$

- Satisfied for $\phi_j = \widehat{\beta}_j \tilde{X}_j$

Feature attribution compared with feature selection for linear regression

- Feature attribution: $\phi_j = \widehat{\beta}_j \tilde{X}_j$
- Feature selection: select if and only if $\widehat{\beta}_j \neq 0$
- Global vs local: selection does not depend on input/instance feature, attribution does

Example: Housing data

Y : House price; X_1 : Square footage; X_2 : Number of bedrooms; X_3 : Neighborhood quality score

$$\mathbb{E}[Y|X_1, X_2, X_3] = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3$$

Data: ($p = 3$ features and $n = 300$ samples)

$$(\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n$$

Prediction algorithm/estimator:

$$\widehat{f}(\cdot) = \mathcal{A}(\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n) = (\widehat{\beta}_1, \widehat{\beta}_2, \widehat{\beta}_3)$$

Housing data co-efficients code

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# Load data
df = pd.read_csv("Lecture_1_HousePrices.csv")

# Separate features and target
X = df.drop(columns=["price"])
y = df["price"]

# Fit model
model = LinearRegression()
model.fit(X, y)

# Store coefficients
feature_names = X.columns
coef = pd.Series(model.coef_, index=feature_names)
intercept = model.intercept_

print("Coefficients:\n", coef.sort_values(ascending=False))
```

```
Coefficients:
neighborhood    21193.399190
sqft            153.549491
bedrooms       -2920.164922
dtype: float64
```

From co-efficients to local and global attribution

- Feature attribution: $\phi_j = \widehat{\beta}_j \tilde{X}_j$ for $j = 1, 2, 3$
- We get *local attribution* for each sample $i = 1, 2, \dots, n$ using the features $(X_1^{(i)}, X_2^{(i)}, X_3^{(i)})$
- We can also get *global attribution* by taking an average

$$\overline{\phi_j} = \frac{1}{n} \sum_{i=1}^n \phi_j^{(i)} = \frac{1}{n} \sum_{i=1}^n \widehat{\beta}_j X_j^{(i)}$$

Housing global attribution

```
: global_attr = np.mean(np.abs(X * model.coef_), axis=0)
  global_attr = pd.Series(global_attr, index=feature_names)

  print(global_attr.sort_values(ascending=False))
```

sqft	261270.328908
neighborhood	16481.498542
bedrooms	9217.987270

dtype: float64

Housing local attribution code

```
import matplotlib.pyplot as plt

sample_indices = [55, 107, 159, 288] # pick 5 examples
local_contribs = pd.DataFrame(index=feature_names)

for idx in sample_indices:
    x_example = X.iloc[idx]
    pred = model.predict([x_example])[0]
    true_val = y.iloc[idx]

    # Local contributions
    contrib = pd.Series(model.coef_ * x_example, index=feature_names).sort_values()

    # Plot
    plt.figure(figsize=(8,5))
    contrib.plot(kind='barh', color=['green' if v>0 else 'red' for v in contrib])
    plt.title(f"Sample {idx}: Prediction = {pred:.2f}, True Value = {true_val:.2f}")
    plt.xlabel("Feature Contribution")
    plt.ylabel("Features")
    plt.tight_layout()
    plt.show()
```

Housing local attribution specific example

```
# Pick a row
idx = 55

x = X.iloc[idx]
pred = model.predict([x])[0]

contributions = coef * x
contributions = pd.Series(contributions, index=feature_names)

print(f"Prediction: {pred:.2f}")
print("\nFeature contributions:")
print(contributions.sort_values(ascending=False))
```

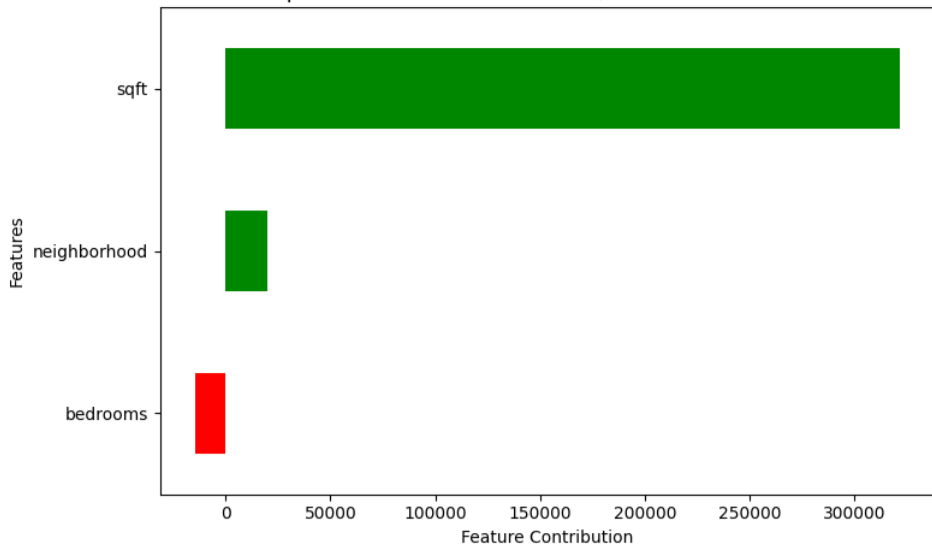
Prediction: 334260.19

Feature contributions:

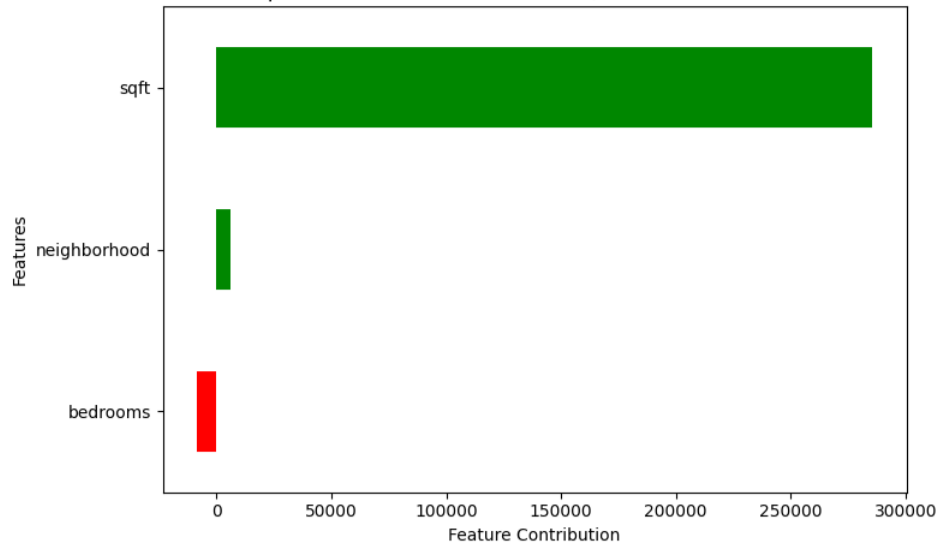
sqft	321353.306080
neighborhood	19736.991322
bedrooms	-14600.824610
dtype:	float64

Housing local visualization example

Sample 55: Prediction = 334260.19, True Value = 386387.16



Sample 288: Prediction = 290696.85, True Value = 266033.18



Summary of housing data results

- In this simple housing data example, global and local attributions reveal relative similar conclusions
- Square footage has the most significant contribution, whereas neighborhood and bedrooms not as significant
- Since independent linear model fits data really well, global vs local not significant
- Fits more into traditional setting, where feature selection and importance (co-efficient values) is sufficient
- Local explanations don't reveal much

Example 2: MNIST image data

- MNIST digit image data
- Binary classification problem, is image digit '7' (or any other digit)
- Features are individual pixels
- Use simple logistic regression model
- Goal is to focus on local feature attribution for individual images (and global attribution)

MNIST logistic regression model

$Y : \{0, 1\}$ (is digit '7'); X_j : intensity at each pixel index j

Model:

$$\mathbb{E}[Y|X_1, X_2, \dots, X_{784}] = f(X_1, X_2, \dots, X_{784}) = 1 - (1 + \exp(\sum_{j=1}^p \beta_j X_j))^{-1}$$

Data: ($p = 28 \times 28$ features and n thousands images)

$$(\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n$$

Prediction algorithm/estimator:

$$\mathcal{A}(\{X_j^{(i)}\}_{j=1}^p, Y^{(i)})_{i=1}^n = (\widehat{\beta}_1, \widehat{\beta}_2, \dots, \widehat{\beta}_{784})$$

Feature attribution for logistic regression

- Feature attribution: $\phi_j = \widehat{\beta}_j \tilde{X}_j$ for $j = 1, 2, \dots, 784$
- We get *local attribution* for each sample $i = 1, 2, \dots, n$ using the features $(X_1^{(i)}, X_2^{(i)}, \dots, X_{784}^{(i)})$
- In this, we simply use *global importance* by looking at the co-efficient values:

$$\phi_j^G = \widehat{\beta}_j$$

MNIST global feature attribution code

```
: import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

# Load MNIST data
mnist = np.load('mnist.npz')
X_train = mnist['x_train'].reshape(-1, 28*28) / 255.0 # flatten and normalize
y_train = mnist['y_train']
X_test = mnist['x_test'].reshape(-1, 28*28) / 255.0
y_test = mnist['y_test']

# Fit a simple one-vs-rest logistic regression for digit "7"
target_digit = 7
y_binary = (y_train == target_digit).astype(int)
model = LogisticRegression(max_iter=500, solver='saga')
model.fit(X_train, y_binary)

# Global feature importance = absolute weights
global_importance = np.abs(model.coef_).reshape(28,28)

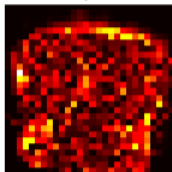
# Pick 5 test samples of this digit
sample_indices = [0, 1, 2, 3, 100,200,300]
fig, axes = plt.subplots(len(sample_indices)+1, 2, figsize=(8, 2*(len(sample_indices)+1)))

# -----
# Global importance plot
# -----
axes[0,0].imshow(global_importance, cmap='hot')
axes[0,0].set_title("Global Feature Importance (weights)")
axes[0,0].axis('off')

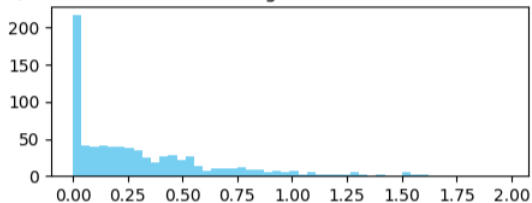
axes[0,1].hist(global_importance.ravel(), bins=50, color='skyblue')
axes[0,1].set_title("Global Weight Distribution")
```

MNIST global feature attribution plot

Global Feature Importance (weights)



Global Weight Distribution



MNIST local feature attribution code

```
# -----  
# Local contributions per sample  
# -----  
for i, idx in enumerate(sample_indices):  
    x = X_test[idx]  
    y_true = y_test[idx]  
    pred_prob = model.predict_proba([x])[0,1]  
    contrib = (x * model.coef_).reshape(28,28) # contribution per pixel  
  
    # Plot local contributions  
    axes[i+1,0].imshow(contrib, cmap='bwr', vmin=-np.max(np.abs(contrib)), vmax=np.max(np.abs(contrib)))  
    axes[i+1,0].set_title(f"Sample {idx} - Pixel Contributions")  
    axes[i+1,0].axis('off')  
  
    # Show the original image  
    axes[i+1,1].imshow(x.reshape(28,28), cmap='gray')  
    axes[i+1,1].set_title(f"Sample {idx} - True: {y_true}, Pred: {pred_prob:.2f}")  
    axes[i+1,1].axis('off')  
  
plt.tight_layout()  
plt.show()
```

MNIST local feature attribution images

Sample 0 - Pixel Contributions



Sample 0 - True: 7, Pred: 1.00



Sample 1 - Pixel Contributions



Sample 1 - True: 2, Pred: 0.00



Sample 2 - Pixel Contributions



Sample 2 - True: 1, Pred: 0.00



MNIST local feature attribution images cont.

Sample 3 - Pixel Contributions



Sample 3 - True: 0, Pred: 0.00



Sample 100 - Pixel Contributions



Sample 100 - True: 6, Pred: 0.00



Sample 200 - Pixel Contributions



Sample 200 - True: 3, Pred: 0.00



Summary and conclusions for MNIST data

- Local attributions much more appropriate since each digit very different
- Global importance not very revealing since it combines all digits
- Note that we use attribution in co-efficient space
- Key property (completeness/efficiency):

$$\sum_{j=1}^p \phi_j = \widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$$

- Is completeness/efficiency satisfied in this case?

Toy language model highlighting negation

Goal: Determine whether the sentence is positive, negative or neutral.

"movie not good" - positive, wrong

- The word "not" is very confusing (negation issue)
- Interpretability reveals contribution of each word to sentiment prediction, "good" very high, "movie" neutral, and "not" very small
- Illustrate with toy neural network model

Quick description of neural network model

$Y : \{0, 1\}$ (positive/negative sentiment); X_j : token/word

Single layer neural network (width 3):

$$\mathbb{E}[Y|X_1, X_2, \dots] = f(X_1, X_2, \dots)$$

where

$$f(.) = 1(\sum_{m=1}^M w_m \sigma(\sum_{j=1}^p E_{jm} X_j) > 0)$$

and $\sigma(.)$ is the ReLu non-linear activation function.

- $M = 3$ in our example, and single layer in our example
- Usually $\{w_m\}_{m=1}^M$ and $\{E_{jm}\}_{j \in \{1, 2, \dots, p\}, m \in \{1, 2, \dots, M\}}$ learned algorithmically
- To simplify our model, choose them either randomly or deterministically

Token attribution in this case

Single layer neural network (width 3):

$$\mathbb{E}[Y|X_1, X_2, \dots] = f(X_1, X_2, \dots)$$

where

$$f(.) = 1(\sum_{m=1}^M w_m \sigma(\sum_{j=1}^p E_{jm} X_j) > 0)$$

and $\sigma(.)$ is the ReLu non-linear activation function.

- In this case use a different common approach related to gradients:

$$\phi_j = X_j \frac{\partial f}{\partial X_j}$$

- Intuition is gradient multiplied by instance attribution to make it local (related to saliency maps, integrated gradients, e.t.c.)

Calculation reveals:

$$\phi_j = X_j \sum_{m=1}^M E_{jm} w_m 1(\sum_{j=1}^p E_{jm} X_j > 0)$$

Token embedding code

```
import numpy as np
import pandas as pd

# -----
# Vocabulary and embeddings
# -----
vocab = ["good", "bad", "not", "movie", "boring", "great"]
vocab_to_idx = {w:i for i,w in enumerate(vocab)}

np.random.seed(0)
d = 3
E = np.random.randn(len(vocab), d)

# -----
# Output weights
# -----
W_out = {
    "positive": np.array([1.5, 0.5, 1.0]),
    "negative": np.array([-1.2, -0.7, -1.0])
}

# -----
# Input sentence
# -----
sentence = ["not", "good", "movie"]
embeddings = [E[vocab_to_idx[w]] for w in sentence]

# -----
# Original linear logits
# -----
def compute_logit_linear(output_word):
    return sum(np.dot(W_out[output_word], e) for e in embeddings)

logits_linear = {w: compute_logit_linear(w) for w in W_out}
pred_linear = max(logits_linear, key=logits_linear.get)
```

ReLU activation layer

```
# -----  
# Now add a ReLU nonlinearity  
# -----  
def relu(x):  
    return np.maximum(0, x)  
  
# Sum embeddings first  
h = np.sum(embeddings, axis=0)  
h_relu = relu(h)  
  
# Compute logits with ReLU  
logits_relu = {w: np.dot(W_out[w], h_relu) for w in W_out}  
pred_relu = max(logits_relu, key=logits_relu.get)  
  
# Token-level contributions for ReLU (approximate attribution)  
# Since ReLU is nonlinear, we attribute by multiplying each embedding by W_out weighted by whether h_relu>0  
# This is a simple "linearization at h" approach  
mask = (h > 0).astype(float)  
contribs_relu = [(word, np.dot(W_out[pred_relu], e * mask)) for word, e in zip(sentence, embeddings)]  
df_relu = pd.DataFrame(contribs_relu, columns=["token", "contribution"])  
  
# -----  
# Print results  
# -----  
print("\n=== ReLU Model ===")  
print("Logits:", logits_relu)  
print("Prediction:", pred_relu)  
print("Token contributions (approximate):")  
print(df_relu)
```

Predictions and feature attribution

=== ReLU Model ===

Logits: {'positive': 7.213323323037797, 'negative': -6.354470257654389}

Prediction: positive

Token contributions (approximate):

	token	contribution
0	not	1.246235
1	good	3.824895
2	movie	2.142193

Summary and conclusion for language example

- The word "not" does not have a high attribution/impact
- Hence prediction is positive and incorrect (negation issue)
- **Issue:** There is an interaction effect between the 'not' and 'good' that is not captured well by this model
- Need more sophisticated model that captures interactions

Summary and limitations for 3 examples

- All feature attribution approaches are local and give intuitively sensible results
- Existing challenges: needs to be local and somehow capture interactions
- Note that all attribution methods are tied to specific model choice and not generic
- Are there better feature attribution approaches?
- Can we come up with something more general?

How do we make feature attribution more general and principled?

- Shapley value framework (to be covered next week)
- General framework for feature attribution measures that satisfy 4 properties/axioms

1. Efficiency

$$\sum_{j=1}^p \phi_j = \widehat{f}(\tilde{X}_1, \tilde{X}_2, \dots, \tilde{X}_p)$$

2. Symmetry

If X_j and X_k are interchangeable then

$$\phi_j = \phi_k$$

3. Dummy

If X_k contains no information about Y

$$\phi_k = 0$$

4. Additivity

If $f = f_1 + f_2$, for all j

$$\phi_j(f) = \phi_j(f_1) + \phi_j(f_2)$$

Linear regression example

Question: Does linear regression with feature attribution $\phi_j = \widehat{\tilde{X}_j \beta_j}$ satisfy the 4 Shapley value axioms?

Will look at this and other examples next week